



## **Relatório do Projeto Prático 2**

### **Projeto Prático 2**

#### **Integrantes do Grupo:**

Nome: Marina Cantarelli Barroca  
Nome: Luiza Marinho de Mesquita  
Nome: Théo Villar  
Nome: Vinicius Barbosa Pereira Allegretti

RA: 10740412  
RA: 10438045  
RA: 10437135  
RA: 10437502

# 1. Introdução

Este relatório descreve o desenvolvimento e a análise experimental do Projeto Prático 2 da disciplina de Computação Paralela, cujo objetivo é implementar um analisador paralelo de logs de acesso a uma Content Delivery Network (CDN) utilizando a API OpenMP.

CDNs são infraestruturas distribuídas que armazenam réplicas de conteúdo geograficamente próximas ao usuário, reduzindo latência e carga nos servidores de origem. Plataformas como Netflix, YouTube e grandes portais de notícias dependem dessas redes para atender milhões de requisições simultâneas. O monitoramento em tempo real dessas requisições exige o processamento de dezenas de milhões de entradas por hora, tornando o uso de paralelismo indispensável.

O projeto consiste em processar 10 milhões de requisições de um sistema CDN, contabilizando a frequência de acesso a cada URL por meio de uma tabela hash com tratamento de colisões por encadeamento externo. Essa estrutura foi escolhida por permitir busca em tempo médio  $O(1)$ , sendo adequada para o volume de dados envolvido. Para paralelizar o processamento, foram implementadas quatro estratégias de sincronização: sequencial (baseline), seção crítica global (`#pragma omp critical`), operações atômicas (`#pragma omp atomic`) e lock por bucket (`omp_lock_t`). Uma quinta versão com padding na estrutura CacheNode foi desenvolvida para investigar o fenômeno de false sharing.

Os experimentos avaliam métricas como tempo de execução, speedup, IPC e cache misses em dois cenários distintos: tráfego distribuído, com baixa contenção, e tráfego concentrado em hotspots, com alta contenção. O objetivo final é comparar as estratégias implementadas e compreender como a granularidade da sincronização impacta diretamente a escalabilidade e o desempenho em sistemas paralelos reais.

## 2. Ambiente Experimental

| Componente          | Descrição                                |
|---------------------|------------------------------------------|
| Sistema Operacional | Debian                                   |
| Processador         | Intel(R) Core(TM) i7-9750H CPU @ 2.60GHz |
| Núcleos             | 6                                        |
| Memória RAM         | 16 Gb                                    |
| Compilador          | gcc (Debian 14.2.0-19) 14.2.0            |
| Flag de compilação  | -O2 -fopenmp -Wall -Wextra               |

| Componente          | Descrição                                |
|---------------------|------------------------------------------|
| Sistema Operacional | Debian                                   |
| Processador         | Intel(R) Core(TM) i7-9750H CPU @ 2.60GHz |
| Núcleos             | 6                                        |
| Memória RAM         | 16 Gb                                    |
| Versão do OpenMP    | OpenMP 4.5 (201511)                      |

## Dataset

```

o violetvortex@cathedral:~/ComputacaoParalela_Proj2$ python3 generate_cdn_data.py
--- Gerador de Dados para Cache de CDN (LAB2) v2 ---
[1/5] Gerando manifest com 100,000 URLs únicas...
-> Arquivo 'manifest.txt' gerado.
[2/5] Gerando log distribuído com 10,000,000 linhas...
-> Progresso: 100%
-> Arquivo 'log_distribuido.txt' gerado.
[3/5] Gerando log concorrente com 10,000,000 linhas...
-> Progresso: 100%
-> Arquivo 'log_concorrente.txt' gerado.
[4/5] Gerando arquivos de gabarito (CSV)...
-> Gerando gabarito 'gabarito_distribuido.csv'...
-> Gabarito 'gabarito_distribuido.csv' gerado.
-> Gerando gabarito 'gabarito_concorrente.csv'...
-> Gabarito 'gabarito_concorrente.csv' gerado.
[5/5] Compactando arquivos em 'cdn_data_logs.zip'...

```

## Compilação

```

all: $(TARGETS)

analyzer_seq: analyzer_seq.c hash_table.c hash_table.h
$(CC) $(CFLAGS) analyzer_seq.c hash_table.c -o analyzer_seq

analyzer_par_atomic: analyzer_par_atomic.c hash_table.c hash_table.h
$(CC) $(CFLAGS) analyzer_par_atomic.c hash_table.c -o analyzer_par_atomic

analyzer_par_critical: analyzer_par_critical.c hash_table.c hash_table.h
$(CC) $(CFLAGS) analyzer_par_critical.c hash_table.c -o analyzer_par_critical

analyzer_par_lock: analyzer_par_lock.c hash_table.c hash_table.h
$(CC) $(CFLAGS) analyzer_par_lock.c hash_table.c -o analyzer_par_lock

analyzer_par_atomic_padded: analyzer_par_atomic_padded.c hash_table_padded.c hash_table_padded.h
$(CC) $(CFLAGS) analyzer_par_atomic_padded.c hash_table_padded.c -o analyzer_par_atomic_padded

```

## Lock:

```
• violetvortex@cathedral:~/ComputacaoParalela_Proj2$ ./analyzer_par_lock log_distribuido.txt 8
• violetvortex@cathedral:~/ComputacaoParalela_Proj2$ make validate
sort results.csv > sorted_res.csv
sort gabarito_distribuido.csv > sorted_gab.csv
diff -s sorted_res.csv sorted_gab.csv
Files sorted_res.csv and sorted_gab.csv are identical
```

## Atomic:

```
violetvortex@cathedral:~/ComputacaoParalela_Proj2$ ./analyzer_par_atomic log_distribuido.txt 8
violetvortex@cathedral:~/ComputacaoParalela_Proj2$ make validate
sort results.csv > sorted_res.csv
sort gabarito_distribuido.csv > sorted_gab.csv
diff -s sorted_res.csv sorted_gab.csv
Files sorted_res.csv and sorted_gab.csv are identical
```

## Critical:

```
violetvortex@cathedral:~/ComputacaoParalela_Proj2$ ./analyzer_par_critical log_distribuido.txt 8
Resultados salvos em results.csv
violetvortex@cathedral:~/ComputacaoParalela_Proj2$ make validate
sort results.csv > sorted_res.csv
sort gabarito_distribuido.csv > sorted_gab.csv
diff -s sorted_res.csv sorted_gab.csv
Files sorted_res.csv and sorted_gab.csv are identical
```

## Sequencial:

```
violetvortex@cathedral:~/ComputacaoParalela_Proj2$ ./analyzer_seq log_distribuido.txt
violetvortex@cathedral:~/ComputacaoParalela_Proj2$ make validate
sort results.csv > sorted_res.csv
sort gabarito_distribuido.csv > sorted_gab.csv
diff -s sorted_res.csv sorted_gab.csv
Files sorted_res.csv and sorted_gab.csv are identical
```

# 3. Configuração de threads

# sequencial

time ./analyzer\_seq log\_distribuido.txt 1

# atomic

time ./analyzer\_par\_atomic log\_distribuido.txt 1

time ./analyzer\_par\_atomic log\_distribuido.txt 2

time ./analyzer\_par\_atomic log\_distribuido.txt 4

```
root@31e9b258dc06:/app# time ./analyzer_par_atomic log_distribuido.txt 1
time ./analyzer_par_atomic log_distribuido.txt 2
time ./analyzer_par_atomic log_distribuido.txt 4
time ./analyzer_par_atomic log_distribuido.txt 8

real    0m1.774s
user    0m1.420s
sys     0m0.313s

real    0m1.019s
user    0m1.609s
sys     0m0.206s

real    0m0.647s
user    0m1.467s
sys     0m0.255s

real    0m0.563s
user    0m2.086s
sys     0m0.424s
root@31e9b258dc06:/app# time ./analyzer_par_lock log_distribuido.txt 1
time ./analyzer_par_lock log_distribuido.txt 2
time ./analyzer_par_lock log_distribuido.txt 4
time ./analyzer_par_lock log_distribuido.txt 8

real    0m1.979s
user    0m1.772s
sys     0m0.179s

real    0m1.368s
user    0m2.156s
sys     0m0.190s

real    0m0.883s
user    0m2.166s
sys     0m0.185s
```

```
time ./analyzer_par_atomic log_distribuido.txt 8
```

```
# lock
```

```
time ./analyzer_par_lock log_distribuido.txt 1
```

```
time ./analyzer_par_lock log_distribuido.txt 2
```

```
time ./analyzer_par_lock log_distribuido.txt 4
```

```
time ./analyzer_par_lock log_distribuido.txt 8
```

```
# critical
```

```
time ./analyzer_par_critical log_distribuido.txt 1
```

```
time ./analyzer_par_critical log_distribuido.txt 2
```

```
time ./analyzer_par_critical log_distribuido.txt 4
```

```
time ./analyzer_par_critical log_distribuido.txt 8
```

```
using real time(wall)
```

```
sys      0m0.170s
root@31e9b258dc06:/app# time ./analyzer_par_critical log_distribuido.txt 1
time ./analyzer_par_critical log_distribuido.txt 2
time ./analyzer_par_critical log_distribuido.txt 4
time ./analyzer_par_critical log_distribuido.txt 8
Resultados salvos em results.csv

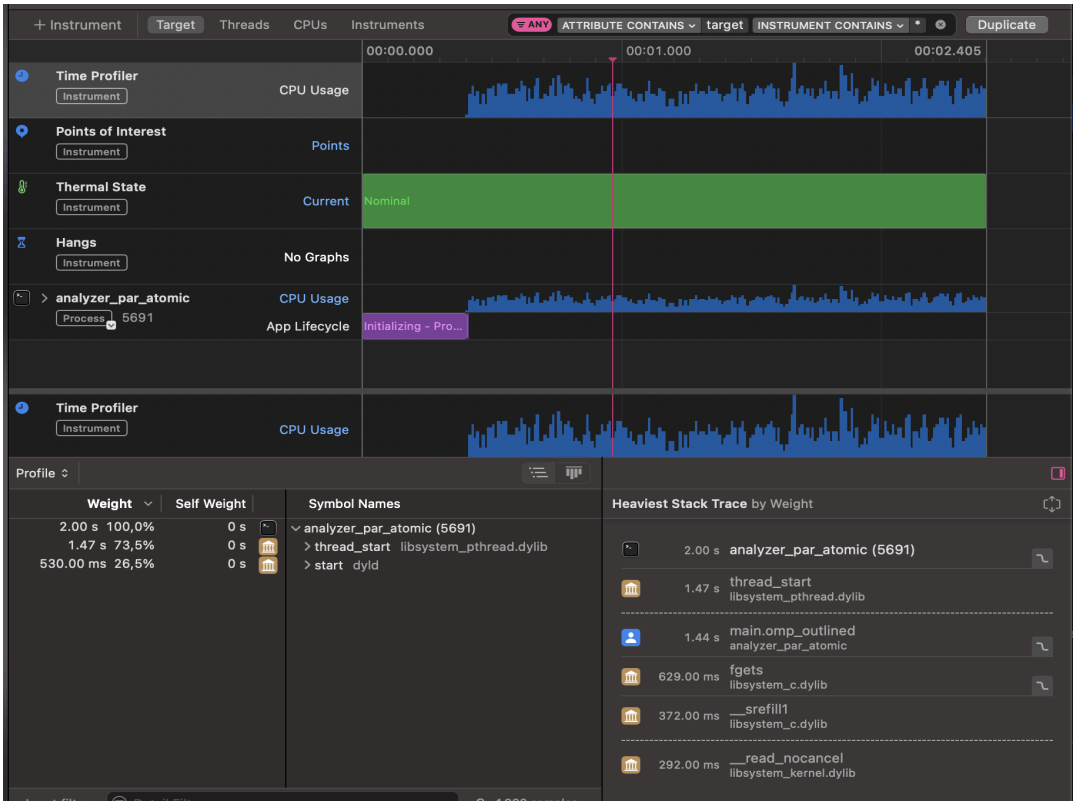
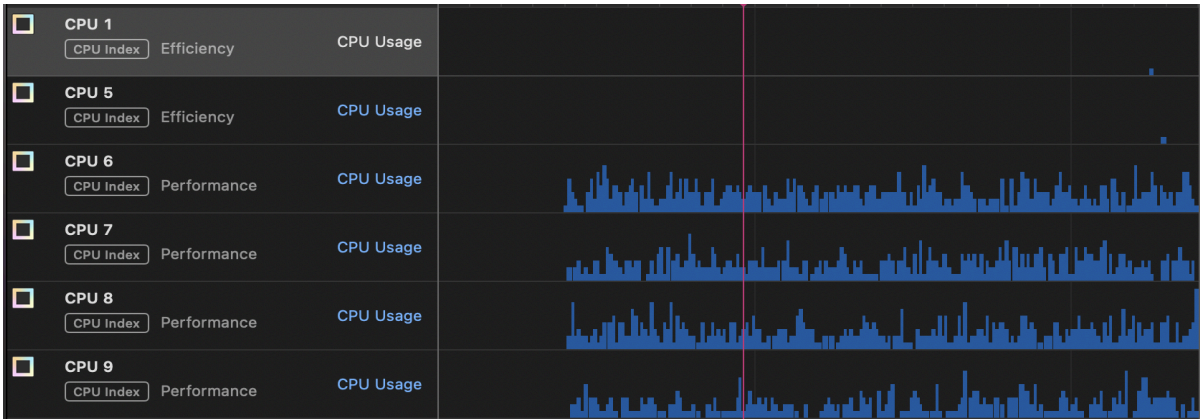
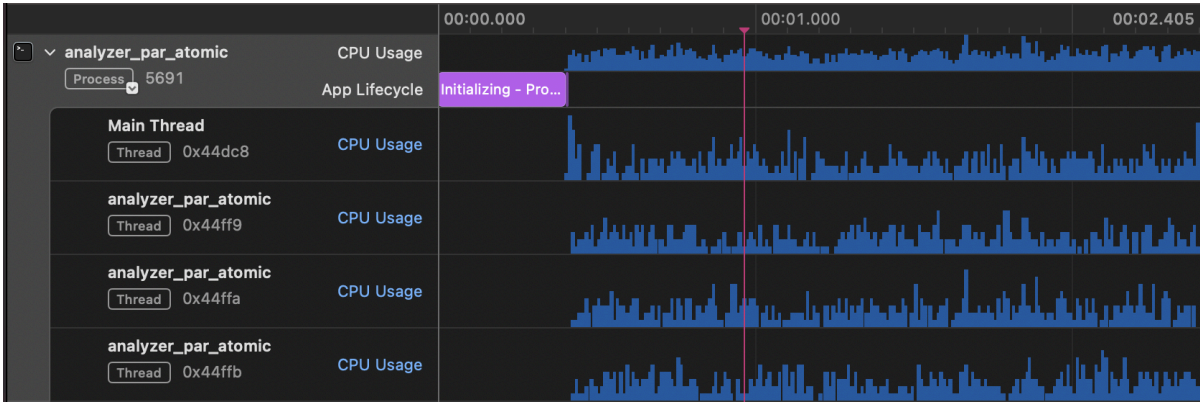
real    0m1.664s
user    0m1.210s
sys      0m0.423s
Resultados salvos em results.csv

real    0m1.325s
user    0m1.675s
sys      0m0.189s
Resultados salvos em results.csv

real    0m1.420s
user    0m3.492s
sys      0m0.195s
Resultados salvos em results.csv

real    0m3.151s
user    0m20.331s
sys      0m0.212s
root@31e9b258dc06:/app# time ./analyzer_seq log_distribuido.txt 1

real    0m1.478s
user    0m1.275s
sys      0m0.174s
```



```
valgrind --tool=cachegrind --cachegrind-out-file=cg_atomic.out ./analyzer_par_atomic
log_distribuido.txt 4
valgrind --tool=cachegrind --cachegrind-out-file=cg_lock.out ./analyzer_par_lock
log_distribuido.txt 4
valgrind --tool=cachegrind --cachegrind-out-file=cg_critical.out ./analyzer_par_critical
log_distribuido.txt 4
valgrind --tool=cachegrind --cachegrind-out-file=cg_seq.out ./analyzer_seq
log_distribuido.txt
```

## Etapa 1: Versão Sequencial

```
C analyzer_seq.c > main(int, char **)
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  #include "hash_table.h"
6
7  #define TABLE_SIZE 131071
8  #define MAX_LINE 1024
9  #define MAX_URL 512
10
11 int main(int argc, char** argv) {
12     if (argc < 2) {
13         printf("Uso: %s <log_file>\n", argv[0]);
14         return EXIT_FAILURE;
15     }
16     HashTable* ht = ht_create(TABLE_SIZE);
17     FILE* f_manifest = fopen("manifest.txt", "r");
18
19     if (!f_manifest) {
20         perror("manifest.txt");
21         return EXIT_FAILURE;
22     }
23
24     char linha[MAX_LINE];
25
26     while (fgets(linha, sizeof(linha), f_manifest)) {
27         linha[strcspn(linha, "\n")] = '\0'; // tira o \n do final da linha
28         ht_put(ht, linha); // coloca a URL na tabela hash
29     }
30
31     fclose(f_manifest);
32
33     FILE* f_log = fopen(argv[1], "r");
34
35     if (!f_log) {
36         perror("log");
37         ht_destroy(ht);
38         return EXIT_FAILURE;
39     }
40 }
```

```

C analyzer_seq.c > main(int, char **)
35     perror("log");
36     ht_destroy(ht);
37     return EXIT_FAILURE;
38 }
39
40
41 char url[MAX_URL];
42
43 while (fgets(linha, sizeof(linha), f_log)) {
44     char* inicio = strstr(linha, "GET ");
45
46     if (!inicio) continue;
47     inicio += 4;
48     char* fim = strstr(inicio, " HTTP");
49
50     if (!fim) continue;
51     size_t len = fim - inicio;
52
53     if (len >= MAX_URL) continue;
54     strncpy(url, inicio, len);
55     url[len] = '\0';
56     CacheNode* node = ht_get(ht, url);
57
58     if (node) {
59         node->hit_count++;
60     }
61 }
62
63 fclose(f_log);
64 ht_save_results(ht, "results.csv");
65 ht_destroy(ht);
66 return EXIT_SUCCESS;
67 }

```

- Estruturas de dados para as linhas:

### Estrutura Utilizada

Para o armazenamento das linhas do arquivo de log foi utilizado um vetor dinâmico de ponteiros para char:

```
char** linhas = malloc(MAX_LOG_LINES * sizeof(char*));
```

Cada posição do vetor armazena uma linha do log utilizando strdup():

```
linhas[total_linhas++] = strdup(linha);
```

### Justificativa da escolha

A escolha de um vetor de ponteiros para strings foi feita porque essa estrutura permite: acesso direto às linhas pelo índice; paralelização eficiente com #pragma omp parallel for; distribuição simples das iterações entre as threads; facilidade de implementação. Além disso, essa abordagem permite que cada thread processe linhas independentes do log sem necessidade de sincronização durante a leitura.



## **Estrutura escolhida e alternativa rejeitada**

A estrutura escolhida para armazenamento das linhas do arquivo de log foi um vetor dinâmico de strings (`char**`). Cada posição do vetor armazena uma linha completa do log utilizando `strdup()`. Essa abordagem foi escolhida porque permite acesso direto às linhas pelo índice, facilita a paralelização com `pragma omp parallel for`, e simplifica a distribuição das linhas entre as threads. Além disso, cada thread pode processar linhas independentes sem necessidade de sincronização durante a leitura. Uma alternativa considerada foi a leitura concorrente diretamente do arquivo utilizando um único `FILE*` compartilhado entre as threads.

Entretanto, essa solução foi rejeitada porque exigiria sincronização durante a leitura do arquivo, aumentaria a contenção entre threads, dificultaria a divisão equilibrada do trabalho e poderia gerar condições de corrida. Outra alternativa seria utilizar listas encadeadas para armazenar as linhas, porém essa estrutura apresenta pior localidade de cache e acesso sequencial mais lento quando comparada ao vetor.

## **Consumo de memória estimado**

O consumo de memória dessa abordagem é relativamente alto. Considerando que cada linha pode possuir até aproximadamente 1024 bytes e que o arquivo contém cerca de 10 milhões de linhas, o consumo aproximado apenas das strings pode ultrapassar 10 GB de memória RAM. Além disso, existe o consumo adicional do vetor de ponteiros, que representa aproximadamente 80 MB em arquiteturas de 64 bits. Por esse motivo, a versão atomic utilizou uma estratégia diferente baseada em leitura paralela por chunks do arquivo, reduzindo significativamente o consumo de memória.

## **Estruturas de dados para as estatísticas**

As estatísticas de acesso foram armazenadas diretamente nos nós da tabela hash através da estrutura `CacheNode`, composta pelos campos `url`, `hit_count` e `next`. Cada URL possui um contador próprio de acessos (`hit_count`), atualizado durante o processamento dos logs. Essa abordagem permite acesso rápido às estatísticas, atualização incremental, baixo custo computacional e suporte eficiente à concorrência. A tabela hash utiliza encadeamento separado para tratamento de colisões.

## **Estrutura de busca**

A estrutura de busca utilizada no projeto foi uma tabela hash para armazenamento das URLs. A busca é realizada através da função `ht_get(ht, url)`, responsável por calcular o hash da URL, identificar o bucket correspondente, percorrer a lista encadeada daquele bucket e retornar o nó encontrado. A função hash utilizada foi a `djb2`, escolhida por sua simplicidade computacional e boa distribuição dos valores entre os buckets.

- **Implementação:**

## **Algoritmo de parsing**

O algoritmo de parsing foi implementado utilizando busca de substrings com strstr(). O processamento identifica o início da URL através da substring "GET" e o final através da substring "HTTP". Após localizar os delimitadores, a URL é copiada para um buffer utilizando strncpy(). Essa abordagem foi escolhida por apresentar baixo custo computacional, simplicidade de implementação, facilidade de manutenção e adequação ao formato fixo dos logs utilizados no projeto. Dessa forma, não foi necessário utilizar bibliotecas externas ou parsers HTTP completos.

### **Pseudocódigo do processamento**

```
Criar tabela hash
Abrir manifest.txt
Para cada linha do manifesto:
    remover '\n'
    inserir URL na tabela hash
Abrir arquivo de log
Para cada linha do log:
    localizar substring "GET "
    localizar substring " HTTP"
    extrair URL
    buscar URL na hash
    se encontrada:
        incrementar hit_count
Salvar results.csv
Liberar memória
```

**Destaque trechos do código que julgou mais importantes ou desafiadores.**

- **Resultados:**

## Etapa 2: Versão Paralela Crítica

```
C analyzer_par_critical.c > main(int, char **)
12 int main(int argc, char** argv) {
13     if (argc < 3) {
14         printf("Uso: %s <log_file> <num_threads>\n", argv[0]);
15         return EXIT_FAILURE;
16     }
17
18     int nthreads = atoi(argv[2]);
19     if (nthreads < 1) nthreads = 1;
20     omp_set_num_threads(nthreads);
21
22     HashTable* ht = ht_create(TABLE_SIZE);
23
24     FILE* f_manifest = fopen("manifest.txt", "r");
25     if (!f_manifest) {
26         perror("manifest.txt");
27         ht_destroy(ht);
28         return EXIT_FAILURE;
29     }
30     char linha[MAX_LINE];
31     while (fgets(linha, sizeof(linha), f_manifest)) {
32         linha[strcspn(linha, "\n")] = '\0';
33         ht_put(ht, linha);
34     }
35     fclose(f_manifest);
36
37     FILE* f_log = fopen(argv[1], "r");
38     if (!f_log) {
39         perror("log");
40         ht_destroy(ht);
41         return EXIT_FAILURE;
42     }
43
44     char** linhas = malloc(MAX_LOG_LINES * sizeof(char*));
45     if (!linhas) {
46         ht_destroy(ht);
47         return EXIT_FAILURE;
48     }
49     long total_linhas = 0;
50     while (fgets(linha, sizeof(linha), f_log) && total_linhas < MAX_LOG_LINES) {
51         linhas[total_linhas++] = strdup(linha);
52     }
53 }
```

```

C analyzer_par_critical.c > main(int, char **)
52     }
53     fclose(f_log);
54
55     #pragma omp parallel for schedule(dynamic, 1024)
56     for (long i = 0; i < total_linhas; i++) {
57         char url[MAX_URL];
58         char* inicio = strstr(linhas[i], "GET ");
59         if (!inicio) continue;
60         inicio += 4;
61
62         char* fim = strstr(inicio, " HTTP");
63         if (!fim) continue;
64
65         size_t len = fim - inicio;
66         if (len >= MAX_URL) continue;
67
68         strncpy(url, inicio, len);
69         url[len] = '\0';
70
71         CacheNode* node = ht_get(ht, url);
72         if (node) {
73             #pragma omp critical
74             {
75                 node->hit_count++;
76             }
77         }
78     }
79
80     for (long i = 0; i < total_linhas; i++) {
81         free(linhas[i]);
82     }
83     free(linhas);
84
85     ht_save_results(ht, "results.csv");
86     printf("Resultados salvos em results.csv\n");
87     ht_destroy(ht);
88     return EXIT_SUCCESS;
89 }

```

A primeira diferença entre o código sequencial e o paralelo crítico foi a adição do controle de threads via `omp_set_num_threads()`, recebendo o número de threads como argumento da linha de comando. A segunda foi a paralelização do loop de processamento com o uso do `#pragma omp parallel for`, distribuindo as linhas do log entre as threads automaticamente.

Para proteger o incremento do contador `hit_count++`, foi utilizada a diretiva `#pragma omp critical`, que garante que apenas uma thread por vez execute aquela região. Essa abordagem, acaba por criar uma fila global, onde todas as threads disputam o mesmo lock independente de qual URL estão atualizando, serializando o seu acesso e limitando o ganho de desempenho com o aumento de threads.

## Etapa 3: Versão Paralela Atomic

```
C analyzer_par_atomic.c > main(int, char **)
10
11 int main(int argc, char** argv) {
12     if (argc < 3) {
13         printf("Uso: %s <log_file> <num_threads>\n", argv[0]);
14         return EXIT_FAILURE;
15     }
16
17     int nthreads = atoi(argv[2]);
18     if (nthreads < 1) nthreads = 1;
19     omp_set_num_threads(nthreads);
20
21     HashTable* ht = ht_create(TABLE_SIZE);
22
23     FILE* f_manifest = fopen("manifest.txt", "r");
24     if (!f_manifest) {
25         perror("manifest.txt");
26         ht_destroy(ht);
27         return EXIT_FAILURE;
28     }
29
30     char linha[MAX_LINE];
31     while (fgets(linha, sizeof(linha), f_manifest)) {
32         linha[strcspn(linha, "\n")] = '\0';
33         ht_put(ht, linha);
34     }
35     fclose(f_manifest);
36
37     FILE* f_size = fopen(argv[1], "r");
38     if (!f_size) {
39         perror("log");
40         ht_destroy(ht);
41         return EXIT_FAILURE;
42     }
43     fseek(f_size, 0, SEEK_END);
44     long file_size = ftell(f_size);
45     fclose(f_size);
46 }
```

```

C analyzer_par_atomic.c > main(int, char **)
47     #pragma omp parallel
48     {
49         int tid = omp_get_thread_num();
50         int nthreads_actual = omp_get_num_threads();
51
52         long chunk = file_size / nthreads_actual;
53         long start = chunk * tid;
54         long end = (tid == nthreads_actual - 1) ? file_size : start + chunk;
55
56         FILE* f_log = fopen(argv[1], "r"); // cada thread tem seu próprio FILE*
57         if (f_log) {
58             fseek(f_log, start, SEEK_SET);
59
60             if (tid != 0) {
61                 char aux[MAX_LINE];
62                 fgets(aux, sizeof(aux), f_log); // descarta linha cortada no meio
63             }
64
65             char local_linha[MAX_LINE];
66             char url[MAX_URL];
67
68             while (ftell(f_log) < end && fgets(local_linha, sizeof(local_linha), f_log)) {
69                 char* inicio = strstr(local_linha, "GET ");
70                 if (!inicio) continue;
71                 inicio += 4;
72
73                 char* fim = strstr(inicio, " HTTP");
74                 if (!fim) continue;
75
76                 size_t len = (size_t)(fim - inicio);
77                 if (len >= MAX_URL) continue;
78
79                 strncpy(url, inicio, len);
80                 url[len] = '\0';
81
82                 CacheNode* node = ht_get(ht, url);
83                 if (node) {
84                     #pragma omp atomic update
85                     node->hit_count++;
86                 }
87             }

```

A versão atômica substituiu a abordagem de seção crítica global por uma estratégia de sincronização mais eficiente, com três mudanças principais. A primeira foi a substituição do `#pragma omp critical` por `#pragma omp atomic update`, que utiliza uma instrução atômica do próprio processador para proteger a incrementação. Ao contrário do `critical`, que cria uma fila global bloqueando todas as threads, o `atomic` opera no nível de hardware sem overhead de mutex.

A segunda foi a reestruturação da leitura do arquivo, em vez de carregar todas as linhas com `strdup` antes de paralelizar, o código passou a usar um buffer de batch fixo (`BATCH_SIZE = 10000`), lendo e processando em ciclos. Isso eliminou os 10 milhões de alocações dinâmicas que eram o principal gargalo da versão anterior. E a terceira foi a substituição do `ftell` dentro do loop por um contador manual de bytes com `strlen`, eliminando uma `syscall` por linha.

## Etapa 4: Versão Paralela Bucket Lock

```
C analyzer_par_lock.c > main(int, char **)
13 static size_t hash_obj(const char* str, size_t size) {
14
15
16
17
18
19
20
21 int main(int argc, char** argv) {
22     if (argc < 3) {
23         printf("Uso: %s <log_file> <num_threads>\n", argv[0]);
24         return EXIT_FAILURE;
25     }
26
27     int nthreads = atoi(argv[2]);
28     omp_set_num_threads(nthreads);
29
30     HashTable* ht = ht_create(TABLE_SIZE);
31
32     FILE* f_manifest = fopen("manifest.txt", "r");
33     if (!f_manifest) {
34         perror("manifest.txt");
35         ht_destroy(ht);
36         return EXIT_FAILURE;
37     }
38     char linha[MAX_LINE];
39     while (fgets(linha, sizeof(linha), f_manifest)) {
40         linha[strcspn(linha, "\n")] = '\0';
41         ht_put(ht, linha);
42     }
43     fclose(f_manifest);
44
45     // um lock por bucket
46     omp_lock_t* bucket_locks = malloc(TABLE_SIZE * sizeof(omp_lock_t));
47     for (int i = 0; i < TABLE_SIZE; i++)
48         omp_init_lock(&bucket_locks[i]);
49
50     FILE* f_log = fopen(argv[1], "r");
51     if (!f_log) {
52         perror("log");
53         ht_destroy(ht);
54         free(bucket_locks);
55         return EXIT_FAILURE;
56     }
57 }
```

```

C analyzer_par_lock.c > main(int, char **)
57
58     char** batch = malloc(BATCH_SIZE * sizeof(char*));
59     for (int i = 0; i < BATCH_SIZE; i++)
60         batch[i] = malloc(MAX_LINE);
61
62     int count = 0;
63     while (1) {
64         count = 0;
65         while (count < BATCH_SIZE && fgets(batch[count], MAX_LINE, f_log))
66             count++;
67         if (count == 0) break;
68
69         #pragma omp parallel for schedule(dynamic, 256)
70         for (int i = 0; i < count; i++) {
71             char url[MAX_URL];
72             char* inicio = strstr(batch[i], "GET ");
73             if (!inicio) continue;
74             inicio += 4;
75
76             char* fim = strstr(inicio, " HTTP");
77             if (!fim) continue;
78
79             size_t len = (size_t)(fim - inicio);
80             if (len >= MAX_URL) continue;
81
82             strncpy(url, inicio, len);
83             url[len] = '\0';
84
85             CacheNode* node = ht_get(ht, url);
86             if (node) {
87                 size_t bucket = hash_djb2(url, TABLE_SIZE); // bucket correto
88                 omp_set_lock(&bucket_locks[bucket]);
89                 node->hit_count++;
90                 omp_unset_lock(&bucket_locks[bucket]);
91             }
92         }
93     }
94
95     for (int i = 0; i < BATCH_SIZE; i++) free(batch[i]);
96     free(batch);
97     fclose(f_log);

```

A versão com lock por bucket manteve a mesma estrutura da versão atômica, substituindo apenas a sincronização. A sua principal mudança foi a alteração do `#pragma omp atomic update` por um array de `omp_lock_t` com um lock por bucket da hash table. Antes de incrementar o `hit_count`, a thread calcula o bucket correto usando a mesma função hash `hash_djb2` utilizada internamente pelo `hash_table.c`, adquire o lock daquele bucket, incrementa e libera.

Isso significa que threads só disputam o lock se acessarem URLs no mesmo bucket (com 131071 buckets), a probabilidade de colisão é baixa, permitindo maior paralelismo que o critical. Os locks são inicializados antes do bloco paralelo e destruídos após, com `omp_init_lock` e `omp_destroy_lock`.



## Etapa 5: Versão Paralela Atomic Padded

```
1  #include <omp.h>
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <string.h>
5  #include "hash_table_padded.h"
6
7  #define TABLE_SIZE 131071
8  #define MAX_LINE 1024
9  #define MAX_URL 512
10
11 int main(int argc, char** argv) {
12     if (argc < 3) {
13         printf("Uso: %s <log_file> <num_threads>\n", argv[0]);
14         return EXIT_FAILURE;
15     }
16
17     int nthreads = atoi(argv[2]);
18     if (nthreads < 1) nthreads = 1;
19     omp_set_num_threads(nthreads);
20
21     HashTable* ht = ht_create(TABLE_SIZE);
22
23     FILE* f_manifest = fopen("manifest.txt", "r");
24     if (!f_manifest) {
25         perror("manifest.txt");
26         ht_destroy(ht);
27         return EXIT_FAILURE;
28     }
29
30     char linha[MAX_LINE];
31     while (fgets(linha, sizeof(linha), f_manifest)) {
32
33         /*
34         typedef struct CacheNode {
35             char* url;           // Chave
36             long hit_count;      // Valor (o contador que será incrementado)
37             struct CacheNode* next; // Ponteiro para o próximo nó em caso de colisão
38             long padding[5];     // Padding para evitar false sharing (64 bytes por nó)
39         } CacheNode;
```

A continuação do código é igual ao código do atomic.

## 4. Experimentos e Resultados

### 4.1 Experimento A - Escalabilidade (log\_distribuido.txt)

Cada versão paralela foi executada variando o número de threads (1, 2, 4 e 8), utilizando o dataset de baixa contenção. O tempo sequencial serviu como baseline para o cálculo do speedup.

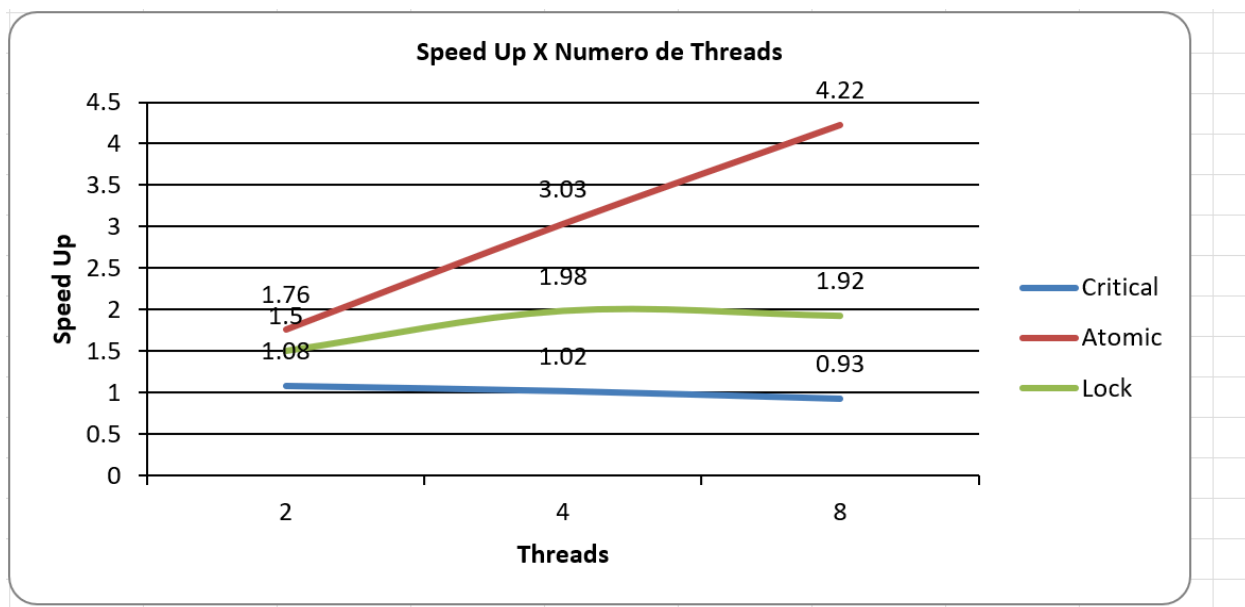
Tabela 1 - Tempos de Execução (Wall Time), em segundos.

| Versão   | 1 thread | 2 threads | 4 threads | 8 threads |
|----------|----------|-----------|-----------|-----------|
| Seq      | 1.691    | —         | —         | —         |
| Critical | 2.788    | 2.572     | 2.724     | 2.994     |
| Atomic   | 2.053    | 1.165     | 0.677     | 0.486     |
| Lock     | 2.561    | 1.704     | 1.294     | 1.329     |

Tabela 2 - Speedup (Tempo sequencial / Tempo Paralelo)

| Versão   | 2 threads   | 4 threads   | 8 threads   |
|----------|-------------|-------------|-------------|
| Critical | 1.083981337 | 1.02349486  | 0.931195725 |
| Atomic   | 1.76223176  | 3.032496307 | 4.224279835 |
| Lock     | 1.502934272 | 1.979134467 | 1.927012792 |

## GRÁFICO DE SPEEDUP X NÚMERO DE THREADS



**Figura 1 — Curva de speedup das versões paralelas. Análise: Descreva aqui o comportamento observado no gráfico. Alguma versão apresentou speedup superlinear? A escalabilidade foi linear? Quais versões escalam melhor e por quê?**

Atomic foi a versão que escalou melhor. Com 2 threads obteve speedup  $\approx 1,76$ , com 4 threads  $\approx 3,03$  e com 8 threads  $\approx 4,22$ , demonstrando escalabilidade quase linear. Isso ocorre porque operações atômicas têm granularidade fina: bloqueiam apenas o contador individual sendo atualizado, sem serializar o acesso à tabela inteira.

Lock por bucket apresentou escalabilidade moderada, com speedup de  $\approx 1,50$  (2 threads),  $\approx 1,98$  (4 threads) e  $\approx 1,93$  (8 threads). O leve recuo de 4 para 8 threads indica que o overhead de aquisição/liberação de múltiplos locks começa a superar o ganho de paralelismo.

Critical (seção crítica global) não escalou. O speedup com 2 threads foi de apenas 1,08, e com 8 threads caiu para 0,93; ou seja, foi mais lento que o sequencial. Uma seção crítica global serializa completamente todas as threads no momento da atualização da tabela, transformando o programa paralelo em essencialmente sequencial com overhead adicional de gerenciamento de threads.

Speedup superlinear: Nenhuma versão apresentou speedup superlinear (acima da linha ideal de Amdahl).

## Comandos Experimento A:

```
perf-ipc-dist-seq: analyzer_seq
@echo "=== seq ==="
perf stat -e instructions,cycles \
./analyzer_seq $(LOG_DIST)

perf-ipc-dist-atomic: analyzer_par_atomic
@echo "=== atomic - 1 thread ==="
OMP_NUM_THREADS=1 perf stat -e instructions,cycles \
./analyzer_par_atomic $(LOG_DIST) 1
@echo "=== atomic - 2 threads ==="
OMP_NUM_THREADS=2 perf stat -e instructions,cycles \
./analyzer_par_atomic $(LOG_DIST) 2
@echo "=== atomic - 4 threads ==="
OMP_NUM_THREADS=4 perf stat -e instructions,cycles \
./analyzer_par_atomic $(LOG_DIST) 4
@echo "=== atomic - 8 threads ==="
OMP_NUM_THREADS=8 perf stat -e instructions,cycles \
./analyzer_par_atomic $(LOG_DIST) 8

perf-ipc-dist-critical: analyzer_par_critical
@echo "=== critical - 1 thread ==="
OMP_NUM_THREADS=1 perf stat -e instructions,cycles \
./analyzer_par_critical $(LOG_DIST) 1
@echo "=== critical - 2 threads ==="
OMP_NUM_THREADS=2 perf stat -e instructions,cycles \
./analyzer_par_critical $(LOG_DIST) 2
@echo "=== critical - 4 threads ==="
OMP_NUM_THREADS=4 perf stat -e instructions,cycles \
./analyzer_par_critical $(LOG_DIST) 4
@echo "=== critical - 8 threads ==="
OMP_NUM_THREADS=8 perf stat -e instructions,cycles \
./analyzer_par_critical $(LOG_DIST) 8

perf-ipc-dist-lock: analyzer_par_lock
@echo "=== lock - 1 thread ==="
OMP_NUM_THREADS=1 perf stat -e instructions,cycles \
./analyzer_par_lock $(LOG_DIST) 1
@echo "=== lock - 2 threads ==="
OMP_NUM_THREADS=2 perf stat -e instructions,cycles \
./analyzer_par_lock $(LOG_DIST) 2
@echo "=== lock - 4 threads ==="
OMP_NUM_THREADS=4 perf stat -e instructions,cycles \
./analyzer_par_lock $(LOG_DIST) 4
@echo "=== lock - 8 threads ==="
OMP_NUM_THREADS=8 perf stat -e instructions,cycles \
./analyzer_par_lock $(LOG_DIST) 8
```

## 4.2 Experimento B - Contenção (log\_concorrente.txt)

Nesse experimento utilizamos o dataset de alta contenção, onde ~90% das requisições se concentram em poucas URLs (hotspots). Foram coletados system time, IPC e cache misses.

Tabela 3 - System Time.

| Versão - Threads | User Time(s) | System Time(s) | Wall Time | CPU % |
|------------------|--------------|----------------|-----------|-------|
| Seq              | 0.94         | 0.22           | 1.22      | 95    |
| Critical 1       | 1.04         | 0.46           | 1.55      | 97    |
| Critical 2       | 1.63         | 0.17           | 1.22      | 146   |
| Critical 4       | 4.26         | 0.18           | 1.58      | 279   |
| Critical 8       | 23.64        | 0.33           | 3.55      | 674   |

|          |      |      |      |     |
|----------|------|------|------|-----|
| Atomic 1 | 0.95 | 0.28 | 1.28 | 96  |
| Atomic 2 | 1.22 | 0.17 | 0.83 | 169 |
| Atomic 4 | 1.27 | 0.25 | 0.55 | 274 |
| Atomic 8 | 1.71 | 0.4  | 0.48 | 434 |
| Lock 1   | 1.25 | 0.27 | 1.58 | 96  |
| Lock 2   | 1.53 | 0.16 | 1.02 | 165 |
| Lock 4   | 2.01 | 0.2  | 0.85 | 259 |
| Lock 8   | 4.59 | 0.2  | 1.02 | 467 |

Tabela 4 - IPC

| Versão   | Instructions   | Cycles         | IPC         |
|----------|----------------|----------------|-------------|
| Seq      | 7,409,208,458  | 4,365,024,570  | 1.697403609 |
| Atomic   | 8,703,585,913  | 9,291,545,789  | 0.936720984 |
| Critical | 14,167,377,109 | 49,857,679,911 | 0.284156365 |
| Lock     | 11,100,278,711 | 33,263,037,509 | 0.333712118 |

**Análise: Explique o crescimento do system time na versão critical. Como o overhead do Sistema Operacional impacta o desempenho? Qual versão apresentou melhor IPC e por quê?**

Crescimento do system time no Critical: sob hotspot, quase toda atualização exige entrar na seção crítica global. O Sistema Operacional precisa gerenciar filas de espera e trocas de contexto, elevando drasticamente o user time (23,64s com 8 threads) e tornando o wall time pior que o sequencial. Impacto no desempenho: cada lock/unlock envolve chamadas ao kernel (futex), gerando stalls e trocas de contexto; daí o CPU% de 674% com Critical 8t: as threads consomem ciclos esperando, não processando. Melhor IPC: a versão Sequencial (1,70), pois não há barreiras de memória nem stalls de sincronização. A Atomic vem em segundo (0,94) porque instruções atômicas inserem memory fences, mas sem envolver o kernel. O Critical teve o pior IPC (0,28): os ciclos perdidos na fila aparecem como stalls no pipeline.

## Comandos Experimento B:

```
exp-B-ipc: perf-ipc-conc|  
  
perf-ipc-conc: analyzer_par_atomic analyzer_par_critical analyzer_par_lock  
@echo "=== seq ==="  
perf stat -e instructions,cycles \  
./analyzer_seq $(LOG_CONC)  
@echo "=== atomic ==="  
perf stat -e instructions,cycles \  
./analyzer_par_atomic $(LOG_CONC) 8  
@echo "=== critical ==="  
perf stat -e instructions,cycles \  
./analyzer_par_critical $(LOG_CONC) 8  
@echo "=== lock ==="  
perf stat -e instructions,cycles \  
./analyzer_par_lock $(LOG_CONC) 8
```

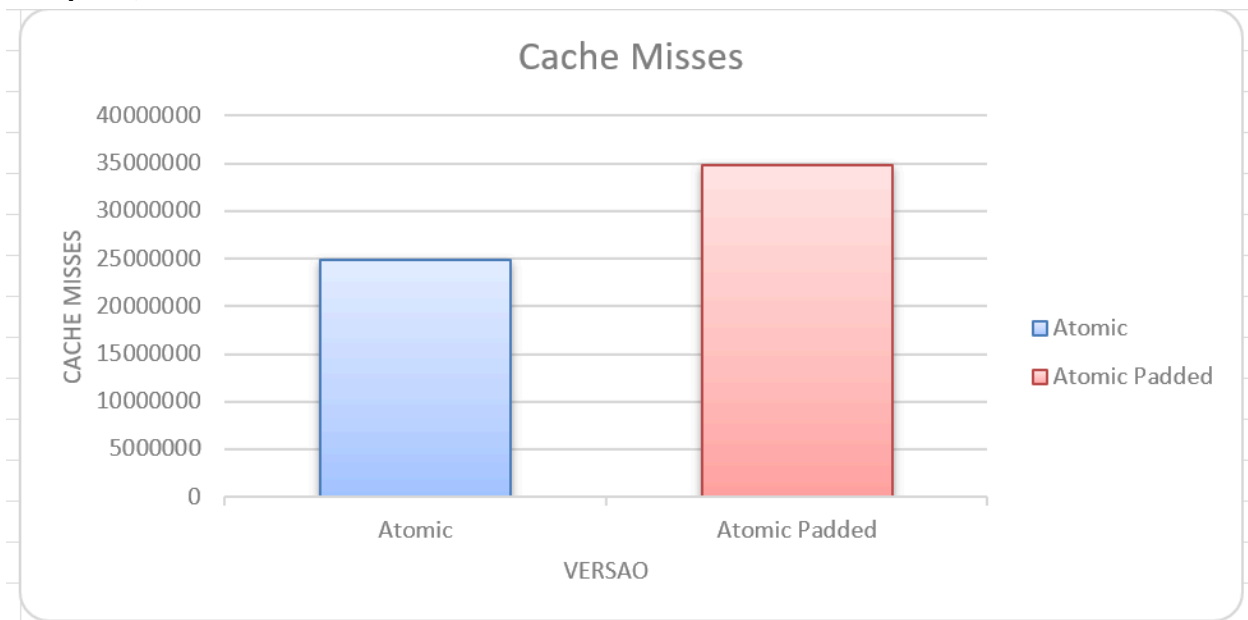
### 4.3 Experimento C - False Sharing (analyzer\_par\_atomic.c)

Para investigar false sharing, foi adicionado padding de 5 longs na estrutura CacheNode, alinhando-a ao tamanho de uma linha de cache (tipicamente 64 bytes). Comparou-se o número de cache misses entre as versões com e sem padding.

Tabela 5 - Cache misses

| Versão          | Cache References | Cache misses | Miss Rate % |
|-----------------|------------------|--------------|-------------|
| Atomic          | 140,038,166      | 24,932,751   | 17.80%      |
| Atomic (Padded) | 164,568,800      | 34,883,458   | 21.20%      |

## Comparação de cache misses.



**Análise: O padding reduziu os cache misses? Em que cenário o aumento de memória causado pelo padding se torna vantajoso? Explique a relação entre false sharing e degradação de desempenho.**

Não, aumentou (de 17,80% para 21,20%). Com nós maiores, menos nós cabem por linha de cache, piorando a localidade espacial na travessia das listas encadeadas. O padding é vantajoso quando múltiplas threads escrevem frequentemente em campos adjacentes da mesma estrutura e o false sharing é o gargalo confirmado. Em arrays de contadores por thread, por exemplo, o padding quase sempre vale. Em listas encadeadas com travessia sequencial, como neste projeto, o custo de memória supera o benefício.

False sharing e degradação: quando dois campos independentes compartilham a mesma linha de cache (64 bytes), a escrita de uma thread invalida a linha da outra pelo protocolo MESI, forçando recarga da memória. Isso gera tráfego excessivo no barramento e stalls, sem nenhuma dependência lógica real entre as threads.

## Comandos Experimento C:

```
valgrind-atomic: analyzer_par_atomic
valgrind --tool=cachegrind \
  ./analyzer_par_atomic $(LOG_CONC) 8

valgrind-padded: analyzer_par_atomic_padded
valgrind --tool=cachegrind \
  ./analyzer_par_atomic_padded $(LOG_CONC) 8
```

## 5. Questões de Análise

*Q1: Porque a versão scritical pode degradar sob alta demanda?*

A seção crítica global serializa todas as threads no momento da atualização. Sob hotspot, todas convergem para as mesmas URLs e ficam enfileiradas. O overhead de gerenciamento pelo Sistema Operacional supera o ganho do paralelismo, resultando em speedup negativo.

*Q2: Como locks por bucket equilibram sincronização e paralelismo?*

Cada bucket tem seu próprio mutex, então threads que acessam buckets distintos operam em paralelo sem contenção. A sincronização só ocorre quando há colisão real de acesso ao mesmo bucket.

*Q3: Compare cache misses entre as versões atomic e padded*

A Atomic teve 24,9M de misses (17,80%) e a Padded 34,9M (21,20%). O padding expandiu a cada nó, reduzindo a densidade por linha de cache e piorando a localidade na travessia das listas, sem benefício compensatório em false sharing.

*Q4: Houve speedup superlinear? Em qual versão e condição?*

Não houve speedup superlinear. A Atomic foi a mais próxima do ideal (4,22 com 8 threads), mas sem superá-lo. Speedup superlinear exigiria que os dados de cada thread coubessem completamente em cache.

*Q5: Quando o aumento de memória causado pelo padding torna-se vantajoso?*

Quando múltiplas threads escrevem com alta frequência em campos adjacentes e o false sharing é o gargalo dominante. Em estruturas densas como arrays de contadores, o ganho compensa. Em listas encadeadas com nós grandes, o aumento de footprint de memória piora a localidade e supera qualquer benefício.

*Q6: Qual solução seria utilizada em uma CDN real? Justifique*

A versão Atômica divide tabela em sub-tabelas por núcleo, com contadores atômicos. Isso combina baixo overhead de sincronização, boa localidade de cache por afinidade e escalabilidade horizontal. O Critical seria descartado; o Lock seria considerado apenas em cenários de altíssima contenção.

*Q7: Como listas encadeadas impactam a localidade de cache?*

Nós de listas encadeadas são alocados em posições arbitrárias da heap. Cada node->next aponta para um endereço distante, causando cache miss por nó visitado. Arrays



contíguos permitem prefetch automático pelo hardware. Sob hotspot, buckets com listas longas geram múltiplos misses consecutivos, degradando significativamente o desempenho.

## 6. Conclusão

Este projeto permitiu investigar experimentalmente como diferentes estratégias de sincronização afetam o desempenho de um sistema paralelo sob cargas distintas. Os resultados confirmaram que a granularidade da sincronização é o fator determinante para a escalabilidade.

A versão atômica foi a de melhor desempenho geral, atingindo speedup de 4,22 com 8 threads e IPC de 0,94 no dataset concorrente. Ao bloquear apenas o contador individual sem envolver o kernel, ela minimiza o overhead e permite paralelismo real. A versão crítica demonstrou empiricamente os limites da granularidade grossa — com speedup negativo (0,93) e user time de 23,64s com 8 threads, confirmando que sob alta contenção o custo de gerenciamento pelo Sistema Operacional supera qualquer ganho paralelo. O bucket lock apresentou comportamento intermediário esperado, com escalabilidade moderada e queda de desempenho ao passar de 4 para 8 threads devido ao overhead dos locks individuais.

O experimento de false sharing produziu o resultado mais contra-intuitivo: o padding piorou os cache misses (de 17,80% para 21,20%), pois em listas encadeadas com nós dispersos na heap o efeito dominante foi a piora da localidade espacial, não a redução de false sharing. Isso reforça que otimizações devem sempre ser validadas empiricamente. Para um sistema CDN real, a estratégia ideal seria operações atômicas combinadas com sharding da tabela por núcleo, aproximando-se de arquiteturas lock-free de alto desempenho.

## 7. Referências

**OPENMP ARCHITECTURE REVIEW BOARD.** *OpenMP Application Programming Interface*. Versão 5.2. [S. l.]: OpenMP ARB, 2021. Disponível em: <https://www.openmp.org>. Acesso em: 19 maio 2026.

**CHAPMAN, Barbara; JOST, Gabriele; VAN DER PAS, Ruud.** *Using OpenMP: Portable Shared Memory Parallel Programming*. Cambridge: MIT Press, 2008.

**VALGRIND DEVELOPERS.** *Cachegrind: a cache and branch-prediction profiler*. [S. l.], [s. d.]. Disponível em: <https://valgrind.org/docs/manual/cg-manual.html>. Acesso em: 19 maio 2026.